

Projet OCR

Nour AMI MOUSSA - Barthélemy BARBIER - Baptiste MAHE
Clément MAUREY

Décembre 2025

Table des matières

1	Introduction	4
2	Projet OCR	4
3	Répartition des tâches	5
3.1	Répartition individuelle	5
3.1.1	Baptiste MAHE	5
3.1.2	Nour AMI MOUSSA	5
3.1.3	Clément MAUREY	5
3.1.4	Barthélemy BARBIER	6
3.2	Répartition générale	6
4	Avancement du projet	7
5	Présentation de l'avancement	8
5.1	INTERFACE	8
5.1.1	Introduction	8
5.1.2	Menu	8
5.1.3	menu.c	9
5.1.4	resolve_window.c	11
5.1.5	Entraînement	14
5.1.6	Résolution d'une grille	16
5.2	AI_NXOR	18
5.3	AI_OCR	21
5.3.1	Introduction	21
5.3.2	Initialisation du réseau et architecture générale	21
5.3.3	Propagation avant et fonction d'acti- vation	22
5.3.4	Calcul de l'erreur et rétro-propagation	22

5.3.5	Paramètres d'apprentissage et critères d'arrêt	23
5.3.6	Sauvegarde des résultats	23
5.3.7	Difficultés rencontrées	23
5.3.8	Conclusion	24
5.4	SOLVER	25
5.4.1	Introduction	25
5.4.2	Fonction loadGrid()	25
5.4.3	Fonction searchWord()	26
5.5	READ_GRID	28
5.5.1	Introduction	28
5.5.2	Suppression des couleurs	28
5.5.3	Rotation de l'image	29
5.5.4	Rotation automatique de l'image . . .	30
5.5.5	Découpage des lettres pour l'OCR . .	33
5.6	Liaison des codes	37
6	Bibliographie	39
7	Conclusion	40

1 Introduction

Pour cette deuxième et dernière soutenance, nous allons vous présenter notre projet sous sa forme finale. Le cahier des charges fourni par l'école nous demandait de terminer le projet **OCR** avec certains critères.

Nous allons aborder les différents éléments que nous avons mis en place sur notre projet afin de répondre au mieux au cahier des charges imposé ainsi que finaliser correctement le projet, en détaillant le mieux possible les différentes étapes de production, en passant par le raisonnement ou encore les difficultés que nous avons eues pendant cette dernière partie du projet.

2 Projet OCR

L'objectif du projet est de réaliser un logiciel de type **OCR** (Optical Character Recognition) qui résout une grille de mots cachés (type mot mêlés) à partir d'une image contenant du texte ainsi qu'une grille, en y ajoutant plusieurs degrés de difficulté (traces, bruit, plis...).

Le projet se divise en deux parties :

- La première partie, qui consiste à effectuer divers traitements ou opérations sur les images pour les rendre exploitables.
- La deuxième partie, qui elle consiste à utiliser les résultats de la première partie et de les exploiter à l'aide d'un réseau de neurones pour identifier les lettres des grilles, et ainsi donc résoudre la grille.

En parallèle de ces deux parties, il est demandé de réaliser des "*preuves de concept*" ou algorithmes de résolution que nous allons vous présenter au fil du rapport de soutenance.

Nous nous sommes donc regroupé en groupe de 4 élèves afin de mener à bien ce projet. Les membres sont : *Baptiste MAHE*, *Nour AMI MOUSSA*, *Clément MAUREY*, *Barthélemy BARBIER*.

3 Répartition des tâches

3.1 Répartition individuelle

3.1.1 Baptiste MAHE

Lors de la préparation de la première soutenance j'ai travaillé sur l'IA booléenne qui doit résoudre un **NXOR**. J'ai créé tout le réseau neuronal et fait tout le fichier **AI_NXOR** de manière plus générale. J'ai aussi travaillé avec Nour afin de mettre en place un algorithme permettant de classifier les patterns et de différencier les lettres dans la grille, les lettres dans les mots à chercher et les patronnes à ignorer.

Lors de la seconde soutenance j'ai travaillé principalement sur l'**AI_OCR**, c'est moi qui ait fait toute la structure du code. J'ai aussi fait toute la partie de l'interface qui gère l'entraînement de l'IA. J'ai aussi travaillé sur le bouton de retour au début du code lorsqu'une image est résolu.

3.1.2 Nour AMI MOUSSA

Pour cette deuxième soutenance, ma principale occupation a été l'interface. Je me suis en effet chargée de toute la structure de celle-ci, la création des fenêtres, des boutons et des titres. Cette architecture de base a été reprise ensuite pour toutes les fenêtres. J'ai également pensé l'apparence de base de cette interface, la manière de positionner les boutons, les images, le choix de couleur ou de police sont tous de ma création.

Ma deuxième tâche était intrinsèquement liée à l'interface puisqu'il s'agissait de mettre en commun les différents codes. En effet, puisque c'est à chaque clique de bouton qu'une fonction se lance, il a semblé logique que la tâche de mise en commun me revienne. J'ai donc conçu notre **main.c** général ainsi que le **Makefile** afin d'appeler toutes les différentes fonctions nécessaires à notre OCR.

3.1.3 Clément MAUREY

Pour cette deuxième soutenance, j'ai principalement concentré mes efforts sur le programme de rotation automatique de l'image. J'ai également travaillé sur l'intelligence artificielle avec Baptiste pour pouvoir finaliser son bon fonctionnement. De plus, j'ai collaboré avec Nour, Barthélemy et Baptiste pour adapter mes programmes afin d'assurer la compatibilité et d'améliorer les fonctionnalités globales de nos différents projets.

3.1.4 Barthélemy BARBIER

Dans le cadre de ce projet de reconnaissance et de résolution automatique de grilles de mots cachés, ma contribution s'est concentrée sur l'intégration entre deux modules essentiels du programme : la détection et la reconnaissance des lettres d'une part (get_letters), et le module de résolution (solver) d'autre part. L'objectif principal de cette partie était de permettre une communication fluide entre ces deux composantes afin d'obtenir un pipeline complet, depuis l'analyse de l'image de la grille jusqu'à la recherche automatisée des mots.

3.2 Répartition générale

	Baptiste	Nour	Clément	Barthélémy
Initialisation	-		+	
Détection lettres		+	-	
Détection mot / grilles			-	+
Réseau de neurones NXOR	+			-
Réseau de neurones OCR	+		-	
Interface graphique		+		-
Compatibilité code	-	+		

[+] : Tâche principale | [-] : Tâche secondaire

4 Avancement du projet

Le tableau ci-dessous présente globalement l'avancement du projet par apport à nos prévisions pour la première soutenance, mais aussi pour les tâches à réaliser pour la soutenance finale en Décembre.

	Prévision	Actuel
Chargement de l'image	100%	100%
Suppression des couleurs	100%	100%
Prétraitement	100%	100%
Détection de la position de la grille	100%	100%
Détection de la position de la liste de mots	100%	100%
Détection des cases de la grille	100%	100%
Récupération sous la forme d'image des lettres et des mots	100%	100%
Reconnaissance des caractères à partir des images	100%	100%
Reconstruction de la grille sous la forme d'un tableau de caractères	100%	100%
Reconstruction de la liste de mots sous la forme d'un tableau de chaînes de caractères	100%	100%
Résolution de la grille	100%	100%
Affichage de la grille résolue	100%	100%
Sauvegarde de la grille résolue	100%	100%

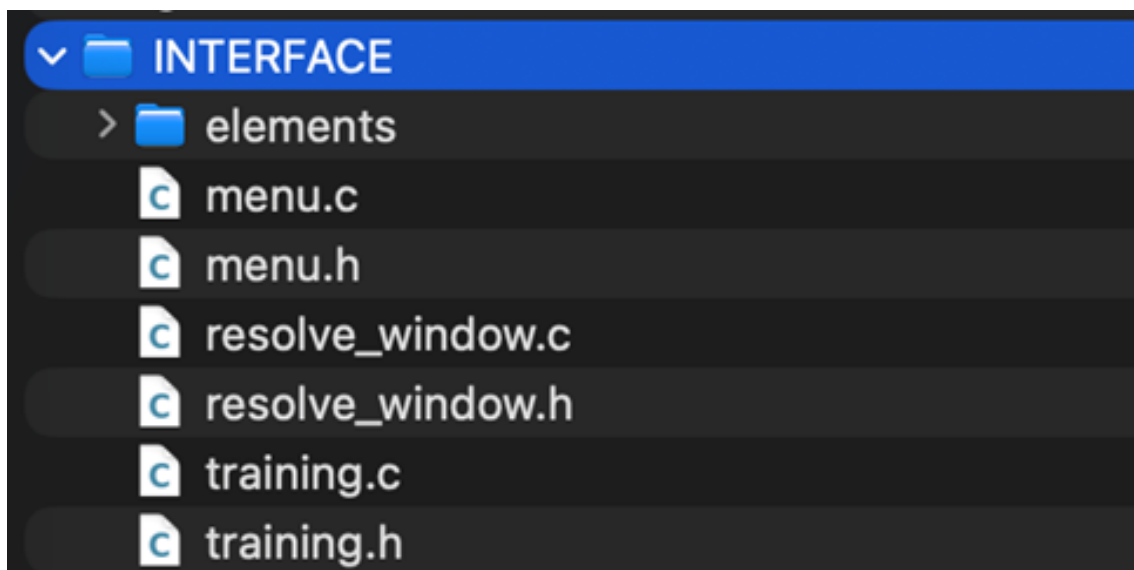
La liste des différentes étapes du projet à été reprise du cahier des charges fourni par EPITA.

5 Présentation de l'avancement

5.1 INTERFACE

5.1.1 Introduction

Ce dossier contient les fichiers **menu.c**, **resolve_window.c**, **training.c**, les headers associés ainsi qu'un sous-dossier **elements**. Le sous-dossier nous sert à stocker les différents éléments utiles à l'affichage tel que les polices d'écriture ou les images de fond utiles aux boutons. Les différents fichiers quant à eux contiennent les codes utilisés pour nos trois grandes fenêtres : le menu, la fenêtre d'entraînement ainsi que celle de résolution de grille.



- Image de l'arborescence de l'interface -

5.1.2 Menu

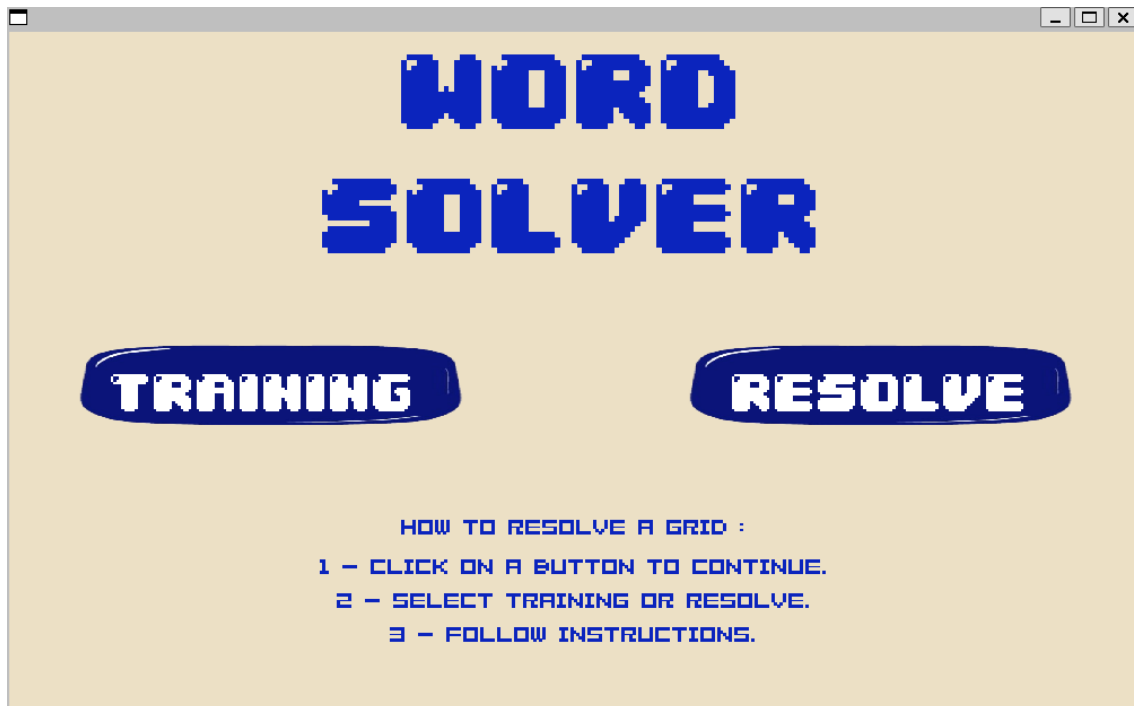
On y retrouve 4 éléments majeurs.

Le premier, éléments contient tous les .png pour rendre notre interface plus jolie et agréable. On y retrouve par exemple le fond de l'interface ou bien même les fonds des boutons.

Le deuxième est menu.c accompagné de son .h . Comme son nom l'indique cette partie représente notre menu, là où l'utilisateur tombera une fois le programme exécuté.

Le troisième est le fichier training.c avec son .c c'est lui qui va s'occuper de l'interface liée à l'IA et plus particulièrement à l'entraînement de cette dernière.

Le dernier est resolve_window.c avec son .h qui est le fichier qui se charge de la sélection de l'image que l'utilisateur souhaite résoudre



- Image du menu de l'interface -

Voici un aperçu de ce que l'on pourra voir grâce à ce menu.

On peut ainsi y voir deux boutons. Un qui mène à une fenêtre pour entraîner notre IA à résoudre les mots cachés en question, et résoudre où l'utilisateur peut choisir une grille que notre programme s'occupera de résoudre.

Comme vous pouvez l'imaginer, `resolve_window.c` et `training.c` s'occupent respectivement de la fenêtre résoudre et de celle du training.

Voyons maintenant plus en détail chacune de ces fenêtres en pénétrant les `.c` et voir leur code respectif.

5.1.3 menu.c

```
void menu_initialize(SDL_Window **window, SDL_Renderer **renderer,  
                    SDL_Rect *training_button, SDL_Rect *resolve_button)  
{  
    (void)window;  
    if (!*renderer || !*renderer)  
        errx(EXIT_FAILURE, "renderer is NULL in menu_initialize");  
  
    SDL_SetRenderDrawColor(*renderer, 236, 224, 197, 255);  
    SDL_RenderClear(*renderer);
```

- Code d'initialisation du menu -

Cette partie nous permet d'initialiser notre fenêtre avec la couleur beige que vous

pouvez retrouver sur la photo précédente.

Ensuite les lignes suivantes nous permettent de placer les mots titre WORD et SOLVER de la taille et la couleur suivante. Les lignes suivantes que vous pouvez retrouver dans notre code nous permet de les mettre aux coordonnées que l'ont veut. `TTF_Font *title_font = TTF_OpenFont("...tittle.TTF", 80); SDL_Color blue = 11, 36, 189, 255;`

```
// ----- buttons -----
SDL_Texture *buttonTex =
    IMG_LoadTexture(*renderer, "INTERFACE/elements/button.png");
if (!buttonTex)
    errx(EXIT_FAILURE, "fail load button.png");

TTF_Font *btnFont = TTF_OpenFont("INTERFACE/elements/tittle.TTF", 40);
if (!btnFont)
    errx(EXIT_FAILURE, "fail open tittle.TTF for buttons");

SDL_Color white = {255, 255, 255, 255};

// TRAINING
SDL_Surface *trainSurf =
    TTF_RenderText_Solid(btnFont, "TRAINING", white);
SDL_Texture *trainTex =
    SDL_CreateTextureFromSurface(*renderer, trainSurf);

SDL_Rect trainRect = {
    (WIDTH - (int)(trainSurf->w * 1.6)) / 2.1,
    200,
    (int)(trainSurf->w * 1.7),
    (int)(trainSurf->h * 4)
};
SDL_RenderCopy(*renderer, buttonTex, NULL, &trainRect);

SDL_Rect trainTextRect = {
    (WIDTH - trainSurf->w) / 2 + 10,
    trainRect.y + (trainRect.h - trainSurf->h) / 2,
    trainSurf->w,
    trainSurf->h
};
SDL_RenderCopy(*renderer, trainTex, NULL, &trainTextRect);
```

- Code de téléchargement des textures -

Ici ce gros passage de code est en réalité très simple. Il nous permet de télécharger nos textures stoches dans éléments, et de pouvoir les mettre dans nos interface. Cette partie aide également à initialiser les boutons, ici celui du training, mais pour résoudre c'est la même chose à des noms de variables près et des coordonnées différentes. Cette partie nous donne la possibilité de modifier la taille des boutons, de leur texte, et de leurs fonds.

```
int menu_event_handler(SDL_Window **window,
                       SDL_Rect *training_button, SDL_Rect *resolve_button)
{
    SDL_Event event;
    (void)window;

    while (SDL_PollEvent(&event))
    {
        if (event.type == SDL_QUIT)
            return 0;

        if (event.type == SDL_MOUSEBUTTONDOWN &&
            event.button.button == SDL_BUTTON_LEFT)
        {
            int mx = event.button.x;
            int my = event.button.y;

            if (isInside(*training_button, mx, my))
                return 3;

            if (isInside(*resolve_button, mx, my))
                return 2;
        }
    }
    return 1;
}
```

- Code de gestion de choix -

La fonction `menu_event_handler` Ici est cruciale pour la gestion du menu. Selon si on clique avec la souris et où on clique cela va nous retourner différentes valeurs qui nous serviront plus tard pour emmener l'utilisateur dans la fenêtre de notre choix. Par exemple si l'utilisateur choisit d'appuyer sur le bouton training, cette fonction va renvoyer 3, qui nous permettra de charger la fenêtre du training. De même pour le bouton resolve.

5.1.4 resolve_window.c

Le fichier `resolve_window.c` implémente l'interface graphique permettant à l'utilisateur de sélectionner une image de grille parmi celles disponibles dans un dossier donné. Cet écran intervient après le menu principal et constitue une étape clé car il prépare notre grille qui sera ensuite traitée par l'algorithme de résolution.

La fonction `count_files`(a retrouve dans notre code) parcourt le répertoire `READ_GRID/images` afin de déterminer le nombre d'images disponibles. Les fichiers cachés (dont le nom commence par un point) sont ignorés.

Ce comptage permet d'adapter l'interface : le nombre d'images affichées à l'écran dépend directement du contenu du dossier. Un maximum de dix images est imposé afin de garantir une interface lisible et éviter toute surcharge visuelle.

La fonction `start_initialize` prépare l'ensemble des éléments graphiques nécessaires à l'affichage de la sélection d'images.

Dans un premier temps, le renderer est nettoyé et rempli avec une couleur de fond. Ensuite, le nombre d'images à afficher est récupéré, puis une disposition en grille est calculée dynamiquement en fonction de ce nombre. Notre programme change si besoin le nombre de lignes et de colonnes, la taille des vignettes, ainsi que leur espacement horizontal.

Cette logique permet d'obtenir une interface flexible selon la situation, capable d'afficher entre une et dix images tout en conservant un rendu cohérent. Dans certains cas particuliers (par exemple lorsque la grille n'est pas complète), un décalage horizontal est appliqué afin de recentrer visuellement les images.

Chaque image est ensuite chargée depuis le disque, convertie en texture SDL, puis associée à un rectangle définissant sa position et sa taille à l'écran. Le nom du fichier image est également stocké afin de pouvoir être affiché ultérieurement sous la vignette correspondante.

```
void start_initialize(SDL_Window *window, SDL_Renderer *renderer,  
    SDL_Texture **texture, ImageItem *images)
```

- Fonction d'initialisation d'image -

À la fin de l'initialisation, seules les informations nécessaires au rendu sont conservées (textures, rectangles, noms), tandis que les chemins temporaires sont libérés afin d'éviter toute fuite mémoire.

La fonction `start_redraw` est responsable de l'affichage complet de l'écran de sélection.

Elle commence par nettoyer le renderer, puis affiche un titre au centre en haut de la fenêtre indiquant à l'utilisateur qu'il doit choisir une image. Si le dossier contient plus de dix images, un message d'avertissement est affiché afin de signaler que seules les dix premières sont visibles.

Ensuite, chaque image est dessinée à l'écran à l'aide de sa texture et de son rectangle associé.

Une fois tous les éléments rendus, la fonction appelle `SDL_RenderPresent`, ce qui rend l'affichage prêt sur l'écran.

Gestion des événements utilisateur (`start_event_handler`)

```
int start_event_handler(SDL_Window *window, ImageItem *images, size_t n)
{
    (void)window;
    SDL_Event event;
    while(SDL_PollEvent(&event))
    {
        switch(event.type)
        {
            case SDL_QUIT :
                return -1;
            case SDL_MOUSEBUTTONDOWN :
                if (event.button.button == SDL_BUTTON_LEFT)
                {
                    int mx = event.button.x;
                    int my = event.button.y;
                    size_t picture = isInsidePicture(images, mx, my, n);
                    if (picture != n+1)
                    {
                        return picture;
                    }
                }
                break;
        }
    }
    return n+1;
}
```

- Code de gestion de clique -

La fonction `start_event_handler` gère les interactions utilisateur sur cet écran.

- la fermeture de la fenêtre (événement `SDL_QUIT`),
- les clics de souris.

Lorsqu'un clic gauche est détecté, la position du curseur est comparée aux rectangles associés aux images affichées. Si le clic se situe à l'intérieur de l'une des images, la fonction retourne l'indice correspondant à l'image sélectionnée. Cette valeur permet au programme principal de savoir précisément quelle grille a été choisie par l'utilisateur.

Si aucun élément n'est cliqué, une valeur spéciale est renvoyée afin d'indiquer qu'aucune sélection n'a été effectuée.

La fonction `isInsidePicture` réalise un test simple permettant de déterminer si un point (correspondant à un clic souris) se situe à l'intérieur du rectangle d'une image.

Cette fonction constitue le cœur de la détection de sélection et permet de transformer une interaction utilisateur en une information exploitable par le programme (sans ça plus rien ne marcherait).

Une fois une image choisie, la fonction `loadChoice` est utilisée pour charger l'image correspondante depuis le disque. Celle-ci est convertie en texture SDL et retournée afin d'être exploitée par les étapes suivantes du programme, notamment pour le traitement de la grille.

Enfin, la fonction `start_clear` libère correctement de toutes les ressources allouées par SDL,

SDL_image et SDL_ttf. Elle détruit la fenêtre, le renderer et ferme proprement les bibliothèques utilisées.

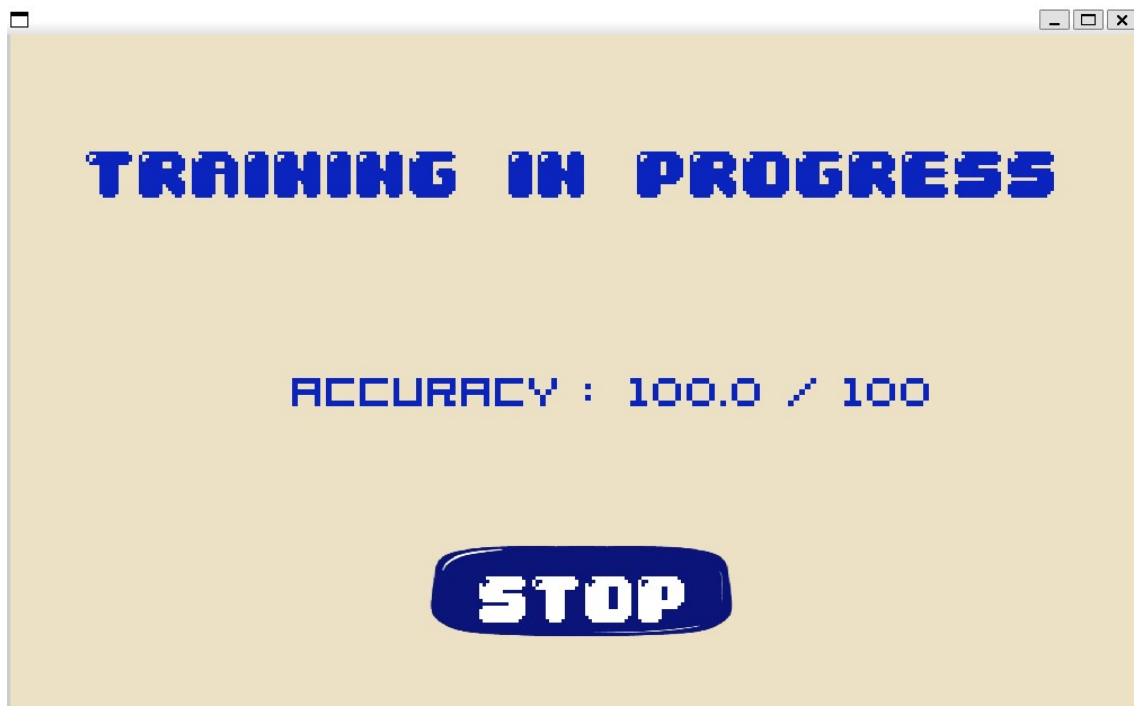
Cette étape garantit une fermeture propre de l'application et évite les fuites mémoire.

```
void start_clear(SDL_Window *window, SDL_Renderer *renderer,  
                SDL_Texture *texture, size_t n, ImageItem *images)  
{  
    if(texture)  
        SDL_DestroyTexture(texture);  
    if(renderer)  
        SDL_DestroyRenderer(renderer);  
    if(window)  
        SDL_DestroyWindow(window);  
  
    TTF_Quit();  
    IMG_Quit();  
    SDL_Quit();  
}
```

- Code de la fermeture du code -

5.1.5 Entraînement

La fenêtre de l'entraînement propose à l'utilisateur de voir la précision de l'IA augmenter en temps réel pendant que cette dernière s'entraîne et d'arrêter l'entraînement au moment où l'utilisateur le décide.



- Fenêtre d'entraînement -

La partie d'entraînement a été un défi singulier dans notre projet car cette fenêtre doit afficher la précision de l'IA tout en entraînant cette dernière, il doit donc être capable d'exécuter deux programmes simultanément car il est nécessaire que le programme soit attentif aux inputs de l'utilisateur pendant l'entraînement pour savoir quand est ce que l'entraînement doit se terminer.

Nous avons donc été contraint de faire tourner plusieurs codes simultanément. C'est pour cette raison que nous avons fait appel à des threads ainsi qu'à un mutex afin de modifier dynamiquement deux valeurs :

- **Accuracy** qui est une valeur qui est lu à chaque instant par le code qui s'occupe de l'affichage et qui est modifié par le code qui s'occupe de l'entraînement de l'IA à chaque fin de cycle forward backward, nous avons donc eu besoin d'utiliser fréquemment le mutex pour ne pas créer de conflit
- **Run** qui est un char que nous utilisons comme un booléen et qui est initialisé à 1 par le code qui gère l'interface puis qui est regardé en permanence par le code qui entraîne l'IA pour savoir si l'entraînement doit continuer. Quand l'utilisateur appuie sur ce bouton, les poids sont enregistrés et cette variable est passée à 0 pour que le code qui entraîne l'IA s'arrête.

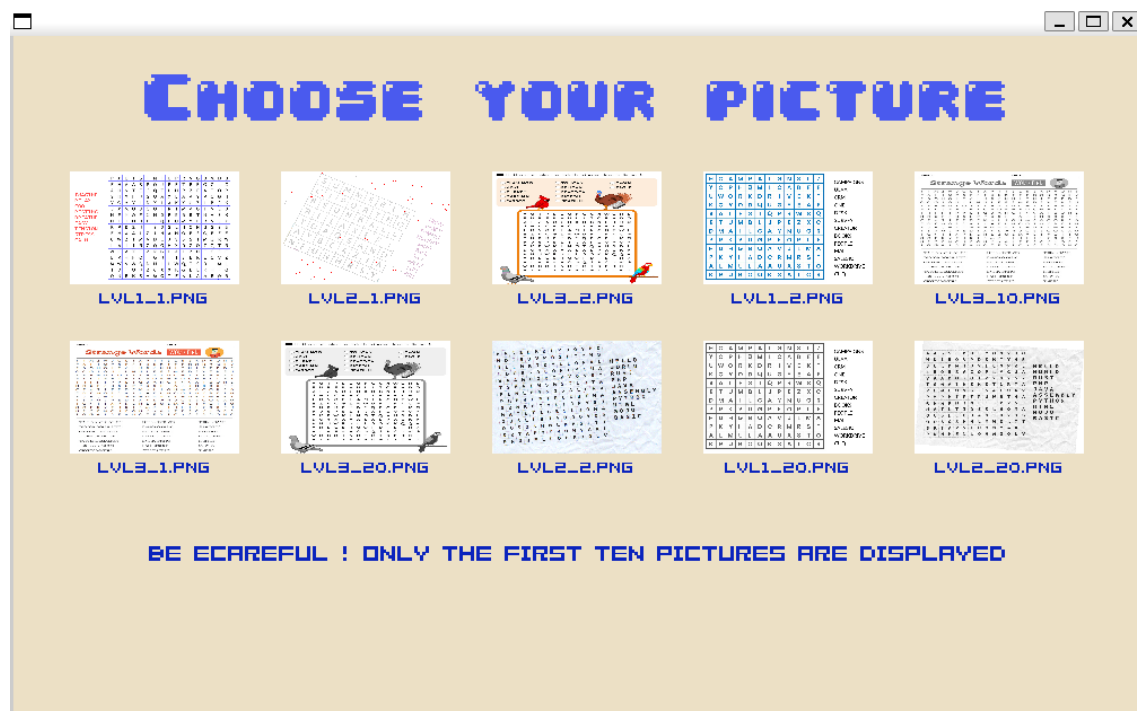
```
64 struct call* c = malloc(sizeof(struct call));
65 if (c == NULL)
66     errx(EXIT_FAILURE, "issue when creating the call");
67
68 float* accuracy = malloc(sizeof(float));
69 if (accuracy == NULL)
70     errx(EXIT_FAILURE, "issue when creating the accuracy");
71 *accuracy = 0;
72
73 char* run = malloc(sizeof(char));
74 if (run == NULL)
75     errx(EXIT_FAILURE, "issue when creating the run");
76 *run = 1;
77
78 pthread_mutex_t *mtx = malloc(sizeof(pthread_mutex_t));
79 pthread_mutex_init(mtx, NULL);
80
81 c->mutex = mtx;
82 c->accuracy = accuracy;
83 c->run = run;
84
85 pthread_t thr;
86 if (pthread_create(&thr, NULL, start_training, (void*)c))
87     errx(EXIT_FAILURE, "issue when creating the thread");
```

- Initialisation du thread et du mutex -

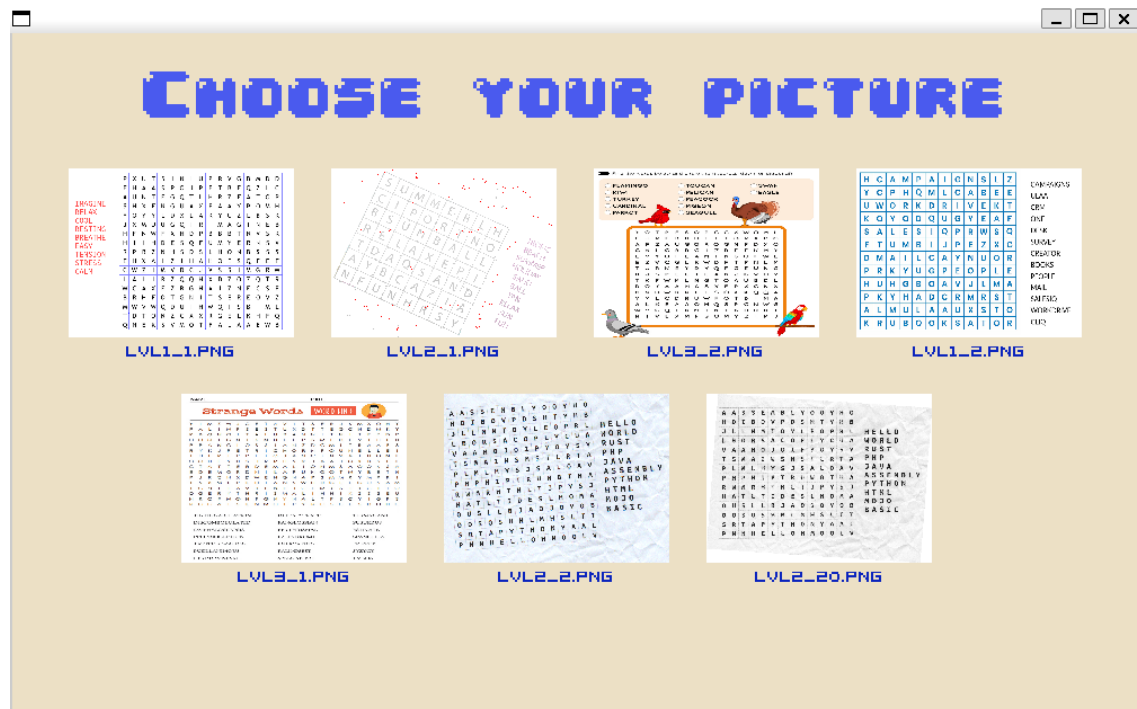
5.1.6 Résolution d'une grille

Après avoir cliqué sur le bouton “resolve”, l'utilisateur a accès à la fenêtre où il effectue le choix de l'image à résoudre. Cette image doit obligatoirement se trouver dans le dossier “images” situé dans le dossier READ_GRID. Le programme va alors parcourir les différents fichiers du dossier et s'il s'agit d'une image, il va l'afficher ainsi que son nom. Pour sélectionner celle que l'utilisateur souhaite résoudre, rien de plus simple, il suffit de cliquer dessus.

Comme dit ci-dessus, le programme va parcourir le dossier images pour afficher toutes les grilles. Ainsi, l'affichage diffère selon le nombre de fichiers. Si on a peu de fichiers, par exemple entre 1 et 3, les images apparaîtront sur une seule ligne et assez grandes. Entre 4 et 10 images, elles seront réparties sur deux lignes différentes. Si le nombre d'images est impair, la première ligne possédera plus d'images que la seconde. Enfin, si le dossier images contient plus de 10 fichiers, seuls les 10 premiers apparaîtront, accompagnés d'un avertissement pour l'utilisateur l'informant que toutes les images ne sont pas visibles sur la fenêtre.



- Sélection parmi plus de dix images -



- Sélection parmi neuf images -

Les principales difficultés de cette partie ont été de rendre dynamique l’affichage des images, le synchroniser avec le dossier contenant les grilles mais aussi de maintenir l’affichage en continu, avec la fenêtre en plein écran ou réduite.

Dans un premier temps, nous avons malheureusement mal interprété la consigne. Dans le cahier des charges, il est demandé de pouvoir charger une image et la résoudre mais pas explicitement dit qu'il pouvait s'agir de n'importe quelle image se trouvant dans un dossier. Nous avons donc d'abord réalisé tout un programme capable d'afficher uniquement les six images mises à notre disposition et de choisir parmi elles. Si le dossier contenait d'autres images, elles ne seraient pas affichées et si à l'inverse il ne contenait pas l'une des six images, le programme s'arrêterait en affichant une erreur. Il nous a donc fallu réécrire toute cette partie ainsi que d'autres fonctions de la main qui partaient du principe qu'il y avait obligatoirement six images lors de la sélection. Toutefois, nous avons pu refaire cette partie sans d'immenses difficultés puisque la logique d'affichage ou la manière d'interagir avec chaque image pour la sélectionner et la charger étaient déjà établies sur notre ancienne version du programme.

5.2 AI_NXOR

Ce dossier contient le fichier **AInxor.c**, **AInxor.h** ainsi que **Makefile** qui permettent respectivement le bon fonctionnement du premier fichier et son exécution.

L'entraînement de l'IA fonctionne en plusieurs étapes. Premièrement il faut mettre en place les couches, les nœuds ainsi que les liens, j'ai choisi de faire trois couches, "*Input Layer*" avec deux nœuds représentant A et B, "*Hidden Layer*" avec deux nœuds et "*Output Layer*" avec un nœud. Ces nœuds et liens sont représentés par des listes de poids et de biais. Une fois cette initialisation faite, le programme va, à chaque itération et pour chaque valeur possible de A et B multiplier les entrées avec les poids des liens en ajoutant les biais des nœuds et en appliquant la fonction sigmoïde afin de s'assurer que la valeur reste entre 0 et 1.

A la fin d'une exécution une valeur de sortie est récupérée et comparée avec le résultat attendu, grâce à la dérivée de la fonction sigmoïde il est possible de calculer l'erreur (à quel point la valeur de sortie est loin de la valeur attendue). Une fois cela calculé, les valeurs des poids et des biais sont plus ou moins modifiés en fonction de la valeur d'erreur et de la variable de vitesse d'apprentissage. Ce programme est exécuté jusqu'à ce que le nombre de génération soit égal à celui défini ou que l'erreur soit suffisamment faible. Il est possible de définir ces variables en les mettant en paramètres.

Ces paramètres peuvent être au nombre de trois, respectivement :

- **Le nombre d'itérations de l'IA**, qui correspond au nombre de corrections maximales que l'IA peut appliquer avant que l'apprentissage se termine. (Comprit dans N^* , 20 000 par défaut)
- **La vitesse d'apprentissage**, qui correspond à un coefficient de correction à chaque itération, s'il est trop faible l'apprentissage sera très lent, s'il est trop grand les poids vont osciller autour de la bonne valeur. (Comprit dans $]0;1]$, 0,6 par défaut)
- **L'objectif d'erreur**, qui correspond au du nombre de chiffres après la virgule de précision qui doit être atteint avant que le programme ne s'arrête car l'erreur est suffisamment faible. (Comprit dans N^* , par défaut 6 (1e-6))

Lors d’une exécution, le résultat est décomposé en plusieurs parties (voir “*CaptureEcranExecution*”).

```
baptiste@DESKTOP-333IMNO:~/OCR_project_spe/AI_NXOR$ ./AInxor
REMEMBER parameters : 1 -> int max_gen (N*) | 2 -> double learning rate (]0,1]) | 3 -> int error_goal (num of nubers after .) (N*)

----- Training -----
-0 | total_error = 0.5284535974 | gen = 0
-1 | total_error = 0.0033483477 | gen = 2000
-2 | total_error = 0.0013322358 | gen = 4000
-3 | total_error = 0.0008199613 | gen = 6000
-4 | total_error = 0.0005893414 | gen = 8000
-5 | total_error = 0.0004588707 | gen = 10000
-6 | total_error = 0.0003751772 | gen = 12000
-7 | total_error = 0.0003170223 | gen = 14000
-8 | total_error = 0.0002743092 | gen = 16000
-9 | total_error = 0.0002416324 | gen = 18000

End: reached 20000 gens. last total_error = 0.00021585

----- Final -----

Final weights :
Input : w_ho[0]=-10.346075 w_ho[1]=-10.328637
Hidden 0 : w_ih[0]=[6.143784, -6.338457] b_h=-3.396258
Hidden 1 : w_ih[1]=[-6.393426, 6.213190] b_h=-3.435664
Output : b_o=5.114974

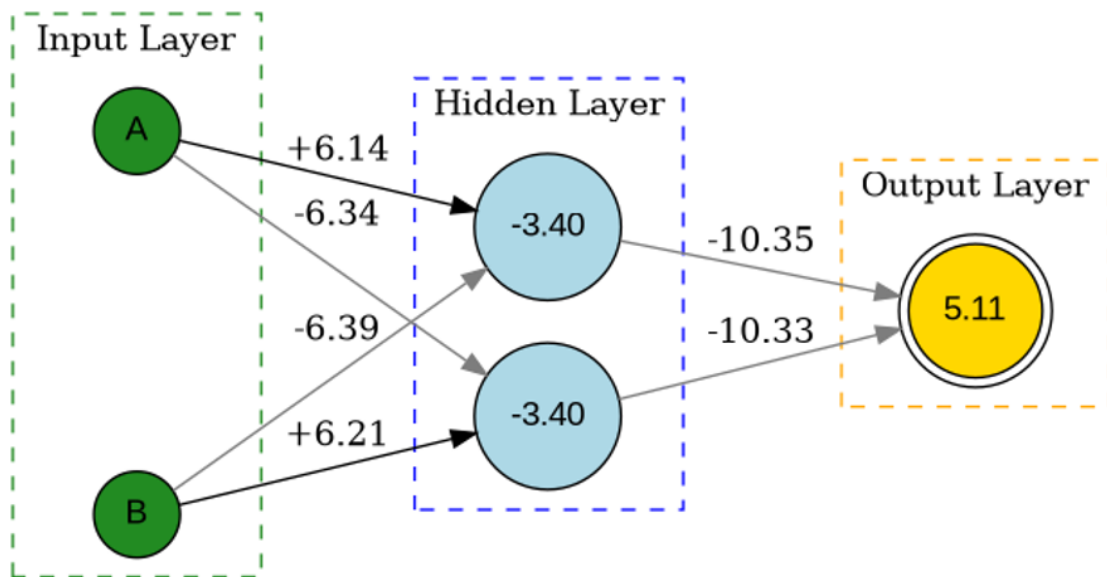
----- Tests -----

Tests ( input {0,1}^2 -> output {0,1} -> rounded {0,1} ) :
0 0 -> 0.988540 -> 1 (success = True)
0 1 -> 0.009858 -> 0 (success = True)
1 0 -> 0.009868 -> 0 (success = True)
1 1 -> 0.989716 -> 1 (success = True)

The file network.png has been generated, open it to see the network
```

- CaptureEcranExecution -

- La première ligne rappelle à quoi correspondent les paramètres. Cette partie contient aussi les messages d’erreur s’il y a trop d’arguments.
- La première partie “*Training*” contient 10 lignes d’entraînement à intervalle régulier, chaque ligne contient l’erreur moyenne de l’IA et la génération à laquelle l’entraînement est. La dernière ligne de cette partie précise la raison pour laquelle l’entraînement s’est arrêté (génération max ou atteinte de la précision voulue).
- La deuxième partie “*Final*” contient une liste des valeurs des poids, des nœuds (b_* , “b” pour “bias”) ou des liens (l_* , “l” pour “link”).
- La troisième partie “*Tests*” contient une liste de tous les tests possibles et des résultats ainsi que la comparaison avec les valeurs attendues (“success” suivi d’un booléen, “True” si le test est réussi, “False” sinon)
- La dernière ligne précise si le .png “*network.pdf*” s’est correctement généré et sinon quelle commande faut-il exécuter pour télécharger .dot (module nécessaire à la génération du png). Ce png contient un graph représentant le réseau neuronal avec les trois couches utilisées et tous les nœuds ainsi que les liens et leurs poids. Un poids négatif est représenté par du gris et un positif par du noir. En noir, les entrées, en bleu les nœuds cachés et en jaune le nœud final. (Exemple de sortie possible : “ExempleNetwork.png”)



- ExempleNetwork -

5.3 AI_OCR

5.3.1 Introduction

Ce dossier contient le fichier **training.c** et **training.h** ainsi que **Makefile** qui sont les fichiers principaux qui permettent le bon fonctionnement du programme.

Le but de cette intelligence artificielle est de reconnaître efficacement les lettres que nous lui donnons sous forme d'image avec une bonne probabilité de succès, dans le but de résoudre efficacement les grilles de mots que nous fournissons au programme.

D'autres fichiers comme **extract_data.c** ou encore **weights.txt** se trouvent aussi dans ce dossier, nous vous expliquerons un peu plus loin leur utilité.

5.3.2 Initialisation du réseau et architecture générale

L'entraînement débute par la mise en place complète du réseau neuronal. Le programme initialise successivement les **couches**, les **nœuds** ainsi que les **liens synaptiques** qui les relient. Contrairement à la première version, qui se limitait à une architecture simple, cette seconde IA repose sur une structure plus élaborée, permettant d'accroître la capacité d'expression et d'apprentissage du modèle.

Chaque couche est composée d'un ensemble de nœuds, chacun muni de poids entrants et d'un biais. L'ensemble de ces paramètres est stocké sous forme de listes, assurant une représentation claire et extensible du réseau. Cette phase initiale garantit une base numérique stable pour les étapes d'entraînement qui suivent. Nous avons décidé de faire quatre couches de trois types différents.

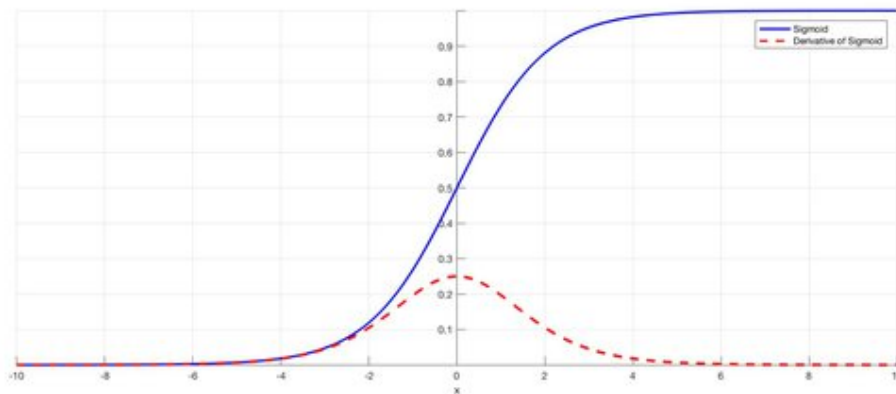
La première est la couche des entrées, elle contient 2058 nœuds et à chaque appel de l'IA l'image d'entrée est vectorisée (transformée en suite de 0 et de 1 représentant les pixels noirs et blancs de l'image) afin que l'IA puisse travailler sur l'image.

La seconde et la troisième couche sont des couches dites "cachées" car l'utilisateur n'interagit pas avec elles, elles sont composées respectivement de 512 et 128 nœuds, le nombre de nœuds est défini de la manière suivante : pour chaque couche n de x nœuds la couche $n+1$ contient environ $x/4$ nœuds. C'est pour cela que nous avons 2 couches cachées car nous devons passer de 2058 à 26 et $44 = 64$ et $64 * 26 = 1664$, deux est donc le nombre de couches cachées qui respecte le mieux cette règle.

La quatrième et dernière ligne est la couche de sortie, elle est composée de 26 nœuds qui prendront des valeurs entre 0 et 1, elle permet de savoir quelle est la lettre qui correspond à l'image. En effet chaque nœud représente une lettre, il y a donc autant de nœuds que de lettres dans l'alphabet et à la fin de l'exécution il nous suffit de choisir le nœud avec la valeur la plus haute et regarder à quelle lettre ce nœud est associé afin d'avoir notre valeur de sortie.

5.3.3 Propagation avant et fonction d'activation

Une fois le réseau initialisé, chaque itération d'apprentissage commence par une phase de propagation avant. Pour chaque entrée présentée à l'IA, les valeurs sont multipliées par les poids associés, puis additionnées aux biais des nœuds concernés. Le résultat passe ensuite par la fonction d'activation sigmoïde, qui contraint chaque sortie dans l'intervalle $[0, 1]$. Cette fonction, également différentiable, est essentielle pour permettre la rétro-propagation du gradient. Lors de cette étape nous utiliserons sa dérivée afin de savoir à quel point la réponse est incertaine.



- Fonction sigmoïde et sa dérivée -

Cette transformation non linéaire permet au réseau de modéliser des comportements complexes et de mieux s'adapter aux variations des données d'entrée.

Voici un résumé des étapes :

1. Chaque pixel de l'image est **multiplié** par un **poids** ;
2. Les résultats sont additionnés avec un **biais** ;
3. La valeur obtenue passe dans une fonction d'activation (ici, une *ReLU* sur les couches internes et un *softmax* en sortie) ;
4. Ce processus se répète de couche en couche.

5.3.4 Calcul de l'erreur et rétro-propagation

À la fin de la propagation, la valeur produite par la couche de sortie est comparée à la valeur attendue. L'écart entre les deux constitue l'erreur du réseau. Grâce à la dérivée de la fonction sigmoïde, l'IA calcule alors le gradient de l'erreur par rapport aux poids et aux biais, ce qui permet d'effectuer les ajustements nécessaires.

La rétro-propagation ajuste ainsi progressivement les paramètres du modèle. Dans cette version améliorée, le processus est plus stable et plus rapide grâce à une gestion affinée des coefficients d'apprentissage.

5.3.5 Paramètres d'apprentissage et critères d'arrêt

L'exécution se poursuit jusqu'à atteindre un critère d'arrêt déterminé par trois paramètres configurables :

- **Nombre maximal d'itérations** : limite le nombre de cycles de correction que l'IA peut effectuer.
- **Vitesse d'apprentissage** : coefficient contrôlant l'amplitude des corrections; une valeur trop faible ralentit la convergence, tandis qu'une valeur trop élevée provoque une instabilité.
- **Objectif d'erreur** : seuil en dessous duquel l'apprentissage s'interrompt automatiquement si la précision souhaitée est atteinte.

Ces paramètres peuvent être fournis en ligne de commande et permettent d'adapter l'apprentissage selon les besoins.

5.3.6 Sauvegarde des résultats

Le fichier **weights.txt** permet de sauvegarder les poids de notre intelligence artificielle et donc de pouvoir réutiliser son apprentissage même après l'avoir relancé. Il s'agit d'un mécanisme important de cette IA car l'apprentissage peut prendre un certain temps avant d'avoir des résultats convenables, et pouvoir sauvegarder son avancement permet un gain de temps considérable. Cette étape de sauvegarde va de paire avec une étape de chargement qui va lire dans le fichier préalablement enregistré et qui va charger les poids et les biais. Il est absolument nécessaire de charger les poids avant toute utilisation de l'IA afin d'avoir des poids déjà entraînées.

5.3.7 Difficultés rencontrées

Une des difficultés majeures que nous avons rencontrées lors de l'entraînement intensif de l'IA était l'apparition de **NaN** dans les poids et dans les sorties du réseau. Lorsque nous essayions d'augmenter le nombre d'itérations ou le taux d'apprentissage pour accélérer la convergence, le réseau devenait instable : la fonction de perte ne diminuait plus, les valeurs explosaient, et les résultats étaient totalement incohérents.

Après des recherches sur Internet, nous avons compris que ce problème était lié à l'**explosion des gradients**. Concrètement, lors de la rétro-propagation, certaines mises à jour des poids pouvaient devenir très importantes, saturant les fonctions d'activation et provoquant l'apparition de valeurs **NaN**.

Pour résoudre ce problème, nous avons implémenté une technique appelée **gradient scaling**. Cette méthode consiste à limiter la valeur maximale de chaque gradient avant de mettre à jour les poids. Ainsi, même lors d'un entraînement plus intensif, les corrections restent *contrôlées* et l'apprentissage devient stable.

Grâce à cette solution, nous avons pu augmenter le nombre d'itérations et le taux

d'apprentissage sans rencontrer de problèmes numériques. Cela a permis au réseau d'apprendre plus efficacement et d'améliorer significativement sa capacité à reconnaître les lettres.

5.3.8 Conclusion

Cette seconde version de l'IA, plus complète et plus robuste, offre une architecture améliorée, un traitement plus stable des gradients et une analyse plus détaillée des résultats. Elle constitue ainsi un outil d'apprentissage et de visualisation nettement plus performant que la version initiale.

5.4 SOLVER

5.4.1 Introduction

Ce dossier contient le fichier **Solver.c** et **Solver.h** ainsi que **Makefile** qui permettent respectivement le bon fonctionnement du premier fichier et son exécution.

Le but de ce programme est de résoudre une grille de mots stockée dans un fichier .txt et de vérifier si un mot passé en paramètre est présent ou non dans cette grille. Il devra donc vérifier à chaque emplacement de la grille si le mot entré en paramètre s'y trouve.

Ce programme a été créé de sorte à vérifier dans plusieurs directions si un mot s'y trouve :

- À droite
- À gauche
- En haut
- En bas
- Diagonale haut-droit
- Diagonale haut-gauche
- Diagonale bas-droit
- Diagonale bas-gauche

Il se décompose en deux fonctions **loadGrid()** et **searchWord()** qui ont chacune des objectifs très spécifiques.

5.4.2 Fonction loadGrid()

Cette fonction présente dans le fichier **Solver.c** permet comme son nom l'indique, de charger la grille présente dans le fichier .txt passé en paramètre. Son utilisation est indispensable pour le bon fonctionnement de l'algorithme car elle vérifie si la grille est valide (Sous la forme d'une matrice rectangle ou carrée), que chaque caractère présent dans la grille est un bien un caractère présent dans l'Alphabet (a-z, A-Z), ou encore qu'elle n'est pas tout simplement vide.

Une fois ces vérifications terminées, la grille présente dans le fichier .txt est découpée caractères par caractères, qui sont eux mêmes placés dans un tableau de taille L*1 (basé sur la taille initiale de la grille dans le fichier). Nous nous retrouvons donc avec un tableau qui contient exactement tous les caractères de la grille.

En cas d'erreur ou de grille invalide, le programme se stoppe et renvoie un message d'erreur adapté en Anglais.

```
swardex@MacBook-Pro-de-Clement OCR % cat test.txt
ABCDEFK
GHIJKL
MNOPQR
STUVWX
YZABCD%

swardex@MacBook-Pro-de-Clement OCR % ./solver test.txt MNO
Line length mismatch.
```

- **Solver.c** avec une erreur générée par **loadGrid()** dû à la taille d'une ligne -

```
swardex@MacBook-Pro-de-Clement OCR % cat test.txt
ABCDEF
GHI^KL
MNOPQR
STUVWX
YZABCD%

swardex@MacBook-Pro-de-Clement OCR % ./solver test.txt MNO
Error: invalid character '^' in grid.
```

- **Solver.c** avec une erreur générée par **loadGrid()** dû à un caractère invalide -

5.4.3 Fonction **searchWord()**

Cette fonction, également présente dans le fichier **Solver.c** permet comme son nom l'indique de rechercher le mot souhaité dans la grille stockée dans le tableau du programme. Elle est appelée uniquement après que la fonction **loadGrid()** ai été exécutée sans erreurs.

Le programme va parcourir chaque caractère présent dans le tableau et vérifier dans toutes les directions citées ci-dessus si une lettre du mot que l'on souhaite trouver ne s'y trouve pas. Il y a plusieurs cas de figure possible :

- La lettre où l'on se trouve n'est pas la première lettre du mot ; on passe à une autre lettre du tableau
- La lettre où on se trouve est la première lettre du mot ; on commence à rechercher les lettres autour dans toutes les directions possibles.

Si une lettre autour est la deuxième lettre du mot, on refait exactement la même manipulation jusqu'à ce qu'on trouve le mot entier, ou que l'on trouve plus de lettre compatibles

Si on ne trouve rien autour de la première lettre, on passe à une autre lettre du tableau jusqu'à ce qu'on en retrouve une, ou qu'on arrive au bout du tableau.

```
swardex@MacBook-Pro-de-Clement OCR % cat test.txt
ABCDEF
GHIJKL
MNOPQR
STUVWX
YZABCD%

swardex@MacBook-Pro-de-Clement OCR % ./solver test.txt MNO
(0,2)(2,2)

swardex@MacBook-Pro-de-Clement OCR % ./solver test.txt MNOQ
Not found
```

- Fonctionnement normal de **Solver.c** -

Ce programme a été très intéressant à mettre en place, le plus difficile à été de gérer tous les cas d'erreur possible que cette fonction pouvait générer entre les caractères valides, le dépassement de taille... La gestion de ces différents cas d'erreur est primordial pour le bon fonctionnement de la fonction et de notre projet.

5.5 READ_GRID

5.5.1 Introduction

Ce dossier contient les différentes fonctions qui nous permettent de gérer le traitement des grilles données sous forme d'image.

Le but de ce programme est d'appliquer différentes modifications à une image donnée en paramètre. Il y a différentes sortes de modifications à appliquer :

- Suppression des couleurs
- Rotation de l'image

Après ces modifications, un autre programme se charge de détecter automatiquement les lettres présentes dans la grille ainsi que les mots à trouver pour résoudre le problème.

5.5.2 Suppression des couleurs

Pour la suppression des couleurs, notre programme parcourt chaque pixel de l'image, récupère ses informations de couleur (r,g,b) et applique une formule de luminance afin de le convertir en une variante de gris ($\text{Uint8 gray} = (\text{Uint8})(0.3 * r + 0.59 * g + 0.11 * b)$) (Norme ITU-R BT.601). Suite à cela, le programme modifie le contraste de l'image pour essayer de faire disparaître les quelques imperfections qui s'y trouve. Une fois cela effectué, le programme va s'occuper de rendre les pixels restants beaucoup plus sombres afin qu'ils soient plus facilement reconnaissables pour la découpe.

H	C	A	M	P	A	I	G	N	S	L	Z	CAMPAIGNS
Y	C	P	H	Q	M	L	C	A	B	E	E	ULAA
U	W	O	R	K	D	R	I	V	E	K	T	CRM
K	O	Y	O	D	Q	U	G	Y	E	A	F	ONE
S	A	L	E	S	I	Q	P	R	W	S	Q	DESK
E	T	U	M	B	L	J	P	E	Z	X	C	SURVEY
D	M	A	I	L	C	A	Y	N	U	O	R	CREATOR
P	R	K	Y	U	G	P	E	O	P	L	E	BOOKS
H	U	H	G	B	O	A	V	J	L	M	A	PEOPLE
P	K	Y	H	A	D	C	R	M	R	S	T	MAIL
A	L	M	U	L	A	A	U	X	S	T	O	SALESIQ
K	R	U	B	O	O	K	S	A	T	O	R	WORKDRIVE
												CLIQ

FIGURE 1 – Avant suppression des couleurs

H	C	A	M	P	A	I	G	N	S	L	Z	CAMPAIGNS
Y	C	P	H	Q	M	L	C	A	B	E	E	ULAA
U	W	O	R	K	D	R	I	V	E	K	T	CRM
K	O	Y	O	D	Q	U	G	Y	E	A	F	ONE
S	A	L	E	S	I	Q	P	R	W	S	Q	DESK
E	T	U	M	B	L	J	P	E	Z	X	C	SURVEY
D	M	A	I	L	C	A	Y	N	U	O	R	CREATOR
P	R	K	Y	U	G	P	E	O	P	L	E	BOOKS
H	U	H	G	B	O	A	V	J	L	M	A	PEOPLE
P	K	Y	H	A	D	C	R	M	R	S	T	MAIL
A	L	M	U	L	A	A	U	X	S	T	O	SALESIQ
K	R	U	B	O	O	K	S	A	T	O	R	WORKDRIVE
												CLIQ

FIGURE 2 – Après suppression des couleurs

Les difficultés de ce programme étaient de le rendre suffisamment stable et précis pour que le résultat soit exploitable pour le découpage des lettres, ce qui à été assez difficile au début et qui a nécessité plusieurs ajustements au fil du temps.

5.5.3 Rotation de l'image

La fonction de rotation parcourt chaque pixel de l'image cible et calcule sa position correspondante dans l'image source en appliquant une transformation géométrique. Pour cela, l'angle de rotation fourni en degrés est d'abord converti en radians afin d'utiliser les fonctions trigonométriques de cosinus et de sinus. Le programme détermine ensuite le centre de l'image source et de l'image résultat afin de réaliser une rotation autour du centre. Pour chaque pixel de l'image résultat, la fonction effectue un *inverse mapping* afin de retrouver la position exacte du pixel dans l'image source. Si les coordonnées calculées se trouvent à l'intérieur de l'image source, la couleur du pixel est copiée ; sinon, le pixel est rempli en noir. Cette méthode permet de générer une image correctement orientée tout en conservant les proportions et les dimensions calculées pour englober l'intégralité de l'image après rotation.

H	C	A	M	P	A	I	G	N	S	L	Z	CAMPAIGNS
Y	C	P	H	Q	M	L	C	A	B	E	E	ULAA
U	W	O	R	K	D	R	I	V	E	K	T	CRM
K	O	Y	O	D	Q	U	G	Y	E	A	F	ONE
S	A	L	E	S	I	Q	P	R	W	S	Q	DESK
E	T	U	M	B	L	J	P	E	Z	X	C	SURVEY
D	M	A	I	L	C	A	Y	N	U	O	R	CREATOR
P	R	K	Y	U	G	P	E	O	P	L	E	BOOKS
H	U	H	G	B	O	A	V	J	L	M	A	PEOPLE
P	K	Y	H	A	D	C	R	M	R	S	T	MAIL
A	L	M	U	L	A	A	U	X	S	T	O	SALESIQ
K	R	U	B	O	O	K	S	A	T	O	R	WORKDRIVE
												CLIQ

FIGURE 3 – Avant rotation de l'image

CLIQ	K	R	U	B	O	O	K	S	A	T	O	R
WORKDRIVE	A	L	M	U	L	A	V	A	U	X	S	T
SALESIQ	P	K	Y	H	A	D	C	R	M	R	S	T
MAIL	H	U	H	G	B	O	A	V	J	L	M	A
PEOPLE	P	R	K	Y	U	G	P	E	O	P	L	E
BOOKS	D	M	A	I	L	C	A	Y	N	U	O	R
CREATOR	E	T	U	M	B	L	J	P	E	Z	X	C
SURVEY	S	A	L	E	S	I	Q	P	R	W	S	Q
DESK	K	O	Y	O	D	Q	U	G	Y	E	A	F
ONE	U	W	O	R	K	D	R	I	V	E	K	T
CRM	Y	C	P	H	Q	M	L	C	A	B	E	E
ULAA	H	C	A	M	P	A	I	G	N	S	L	Z
CAMPAIGNS												

FIGURE 4 – Après rotation de l'image (180 degrés)

La difficulté dans ce programme à été d'appliquer le principe de la trigonométrie sur les pixels d'une image, mais aussi de gérer les pixels qui débordent de l'image et d'adapter la taille de l'image si besoin. Le concept était assez difficile à imaginer et à nécessite beaucoup de documentation mais une fois maîtrisé, à été assez simple à mettre en place

5.5.4 Rotation automatique de l'image

La fonction `detect_rotation_angle_projection` a pour objectif d'estimer l'angle de rotation d'une image contenant du texte ou des lignes horizontales. Elle repose

sur une méthode de projection, consistant à analyser l'alignement des pixels selon différents angles compris entre -45° et 45° .

La fonction commence par vérifier que l'image est valide. Afin de réduire le temps de calcul, une version sous-échantillonnée de l'image est créée en ne conservant qu'un pixel sur quatre dans chaque direction. Chaque pixel est converti en niveau de gris, puis transformé en pixel binaire : un pixel est considéré comme *noir* si sa valeur de gris est inférieure à 128, sinon il est traité comme *blanc*. Cette étape produit une image binaire de taille réduite, suffisamment représentative pour l'analyse.

La fonction explore ensuite tous les angles entre -45° et 45° , avec un pas défini par l'utilisateur (paramètre `step_deg`). Pour chaque angle, une projection est calculée. Plus précisément, on effectue une rotation virtuelle de l'image et, pour chaque pixel noir, on incrémente un accumulateur correspondant à sa position projetée dans cette direction.

Cela donne un vecteur de projections représentant la densité de pixels alignés selon cet angle.

Pour chaque angle testé, un score est calculé en effectuant la somme des carrés de tous les éléments du vecteur de projections. Cette mesure favorise les angles où les lignes de texte produisent des pics nets dans la projection. Un score élevé indique donc une forte structuration horizontale de l'image dans la direction considérée.

L'angle ayant obtenu le score le plus élevé est conservé comme estimation de la rotation de l'image. Il correspond à la direction dans laquelle le texte apparaît le plus droit. La fonction retourne finalement cet angle, exprimé en degrés.

```
swardex@MacBook-Pro-de-Clement Initialisation % ./init lvl2_1.png 1
Angle détecté : -25.00°
```

FIGURE 5 – Angle détecté par la fonction

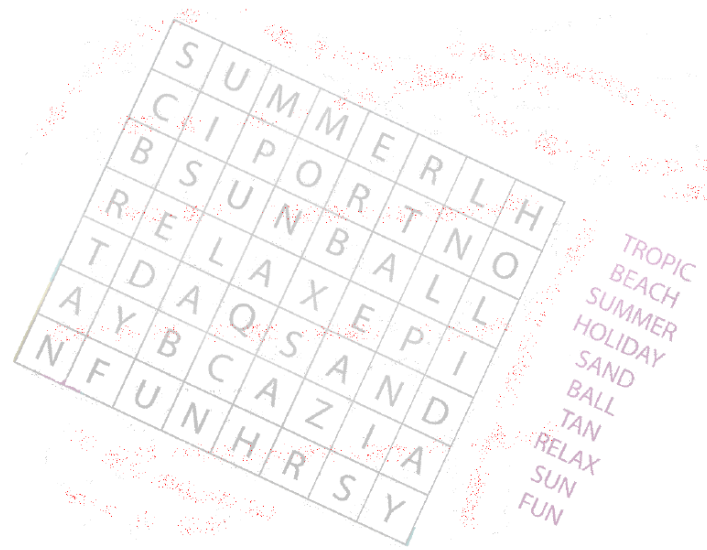


FIGURE 6 – Image utilisée en argument

Les principales difficultés de ces fonctions ont été de visualiser la manière dont le programme pouvait reconnaître si une image était droite ou non. Utiliser les lignes et pixels de l'image pour résoudre ce problème n'a pas été très intuitif au début et a suscité beaucoup d'essais et de réflexion pour arriver à ce résultat. L'utilisation des projections a nécessité beaucoup de recherche pour pouvoir l'implémenter efficacement et arriver à ce résultat.

Mais avec un peu de persévérance j'ai réussi à obtenir un résultat plus convenable qui permet de trouver le bon angle de correction sur toutes les images que nous devons tester.

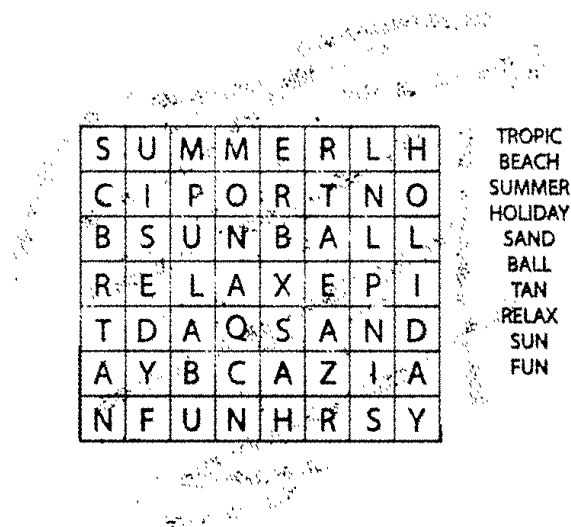


FIGURE 7 – Résultat de la rotation automatique

5.5.5 Découpage des lettres pour l'OCR

Pour le découpage des lettres, notre programme définit d'abord une structure Cell représentant chaque lettre détectée dans une image. Chaque cellule possède les coordonnées maximales x et y pour représenter ses coins, un label pour la différencier des autres cellules, et un attribut family qui indique si elle doit être ignorée, si elle appartient à la grille, ou si elle fait partie d'un mot. Un tableau labels de même dimension que l'image permet de vérifier si chaque pixel noir est déjà labellisé et de l'inclure dans la cellule correspondante. L'algorithme parcourt toute l'image : lorsqu'un pixel foncé n'appartient à aucune cellule, un nouveau label est attribué. Si le tableau de cellules atteint sa capacité, il est réalloué avec le double de sa taille. Un parcours en profondeur récursif inclut alors tous les pixels noirs adjacents dans la cellule, mettant à jour ses attributs. Ce processus permet de former des blobs représentant les différents patrons présents dans notre image.

Les cellules trop petites pour contenir une lettre sont supprimées, et celles qui contiennent une cellule plus petite dans elles même voient leur attribut family modifié pour être ignorées (par exemple pour la grille). Les patrons qui sont trop larges pour être des lettres sont considérés comme des lettres collés et séparés en sous patrons plus petits. Ils sont coupés au niveau des colonnes contenant le plus de blanc car quand les lettres sont collées il n'y a que quelques pixels les rattachant.

Après détection, les cellules sont triées selon leurs coordonnées y puis x afin d'obtenir les lettres ligne par ligne, de gauche à droite. Une fois que tous les patrons de l'image sont connus un algorithme de création de famille est lancé. La première étape de

cet algorithme est de définir une fonction de calcul de distance. Cette première fonction va permettre de comparer deux paternes afin de savoir à quel point ils sont différents, cette fonction inclut la différence de hauteur, de largeur ainsi que la distance euclidienne entre leurs centres respectifs.

Une fois cette fonction crée chaque paterne va créer une famille avec son voisin (le pattern le plus proche selon la fonction distance crée plus tôt). Puis une progression de proche en proche est faite, un seul est défini en fonction de la distance entre tous les patterns de la famille, ce seuil est évolutif et devient de plus en plus stricte dans le temps, il permet de savoir si un pattern peut être inclut dans la famille ou s'il est trop différent. Le caractère évolutif de se pattern a été pensé car deux lettres peuvent avoir une grande différence de largeur (un I et un W) par exemple mais une famille de 100 lettres aura une plus petite différence de largeur entre sa lettre médiane et la nouvelle lettre à inclure.

A chaque itération de notre algorithme de proche en proche, chaque famille regarde quel est le pattern qui est le plus proche de l'un des éléments de la famille et si la distance entre la famille et cet élément est en dessous du seuil de la famille alors le nouvel élément est inclut dans la famille. Sinon la famille est marquée comme complète. Durant le processus nous nous assurons qu'aucune famille n'apparait en double afin de ne pas faire deux fois les calculs. Une fois que toutes les familles sont complètes, nous considéreront que la famille qui contient le plus grand nombre d'élément est la grille. Une fois que la grille est définie il ne reste plus qu'à définir ce qui est une famille parasite et ce qui est un mot.

Plusieurs couches de suppression ont été mises en place afin de supprimer les éléments parasites :

- Toute famille contenant un élément de la grille est une famille parasite qui s'est rattachée à la grille, tous ses éléments n'appartenant pas à la grille sont donc marqués comme parasites.
- Tout élément ayant une taille trop loin de la médiane des tailles de tous les éléments n'appartenant pas à la grille sont considérés comme parasites.
- Tout élément étant trop éloignée (en distance euclidienne) de la médiane de position de tous les éléments n'appartenant pas à la grille sont considérés comme parasites. - Toute famille contenant un parasite est détruite et tous les éléments se retrouvant sans famille après la destruction de cette dernière est considéré comme un parasite.



FIGURE 8 – Image avant découpage

Une fois que tous les éléments parasites ont été marqués il ne nous reste que des lettres dans des mots et des lettres dans la grille. Nous enregistrons donc les lettres de la grille dans un dossier « grid » en les renommant avec un numéro. Les lettres dans des mots sont enregistrées dans un dossier « letters » et renommées avec le numéro de leur famille et un numéro propre (par exemple « word_1_letter_2.png » pour la deuxième lettre du premier mot. Nous n'oublions pas de communiquer les chemins et positions de toutes les lettres ainsi que l'élément auquel elles appartiennent dans l'ordre à la fonction principale afin de faciliter le travail de cette dernière et d'avoir les bons mots ainsi que la taille des lignes de la grille.

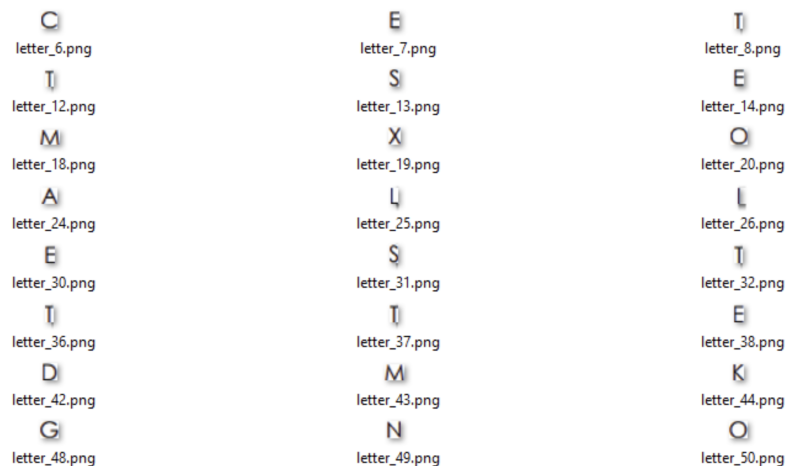


FIGURE 9 – Extrait du résultat après découpage

Difficultés rencontrées :

- La mise en commun des différents programmes : Au début, nous ne voulions pas perdre de temps, nous avons donc commencé à coder la fonction de découpage avant d’avoir la fonction qui rend en noir et blanc l’image fonctionnelle. Nous disposions donc d’une formule temporaire qui donnait des niveaux de gris de chaque pixel pour identifier ce qui doit être ignoré ou non. Arrivé le moment de mettre en commun les différents codes, ils ne fonctionnaient pas de la même manière et la détection s’est donc confrontée à plusieurs problèmes. Nous avons dû reprendre à la fois la fonction de binarisation et celle de détection des cellules, ce qui a coûté un certain temps.
- Séparation des lettres très proches : Certains mots (notamment ceux de la liste) avaient des lettres pratiquement collées les unes des autres. Pour pouvoir correctement les séparer, nous avons dû repasser plusieurs fois sur le code de détection de fin de cellules, nous avons choisi de les séparer sur les colonnes contenant le plus de blanc considérant que les liens entre les lettres sont très fins.
- Différenciation des lettres et éléments extérieurs : Dans les grilles de niveaux supérieurs, beaucoup d’éléments venaient perturber le découpage. Malgré le changement de couleur en noir et blanc, certains parasites subsistaient et formaient des cellules de tailles similaires à celles de lettres. Il nous aura donc fallu créer des tris successifs basés sur la distance et la taille des lettres pour éliminer ces éléments superflus. L’élimination des parasites est probablement le plus gros défi que nous avons rencontré dans ce projet, il nous a demandé beaucoup de temps et de nombreuses étapes afin de s’assurer que tous les cas sont bien gérés sans perdre de lettre dans l’opération

5.6 Liaison des codes

Après avoir réalisé l'ensemble des codes nécessaires à notre OCR, il a enfin fallu écrire le fichier **main.c** principal qui appelle et gère l'ensemble du projet.

Dans un premier temps, cette main va créer une seule et unique fenêtre qui va être passée en paramètre lors de l'initialisation des différents programmes liés à l'interface. Elle va également initialiser le renderer utilisé.

Le premier appel va être l'initialisation du **menu.c**. En effet, c'est le premier affichage auquel va se confronter l'utilisateur et c'est celui qui va permettre de naviguer entre la phase d'entraînement de l'IA et la résolution d'une grille. Suite à cela, il va falloir constamment vérifier sur quel bouton appuie l'utilisateur pour savoir s'il souhaite entraîner l'IA ou non. Si c'est le cas, alors on appelle l'unique fonction du **training.c** et l'entraînement de l'IA va donc se poursuivre jusqu'à ce que l'on décide de l'arrêter. Dans ce cas là, on arrive de nouveau sur le menu principal.

Maintenant si on souhaite résoudre une grille, on appuie sur le bouton correspondant afin de charger la fenêtre de sélection d'image. On continue d'afficher cette fenêtre jusqu'à ce que l'utilisateur choisisse une image à résoudre. A noter que si on ferme la fenêtre, tout le programme s'arrête.

Une fois l'image choisie, celle-ci s'affiche en plein écran et là vient le moment d'appeler toutes les fonctions de résolution. Dans un premier temps, il faut réaliser le pré-traitement de l'image. Les fonctions de binarisation et de rotation automatique du **init.c** sont donc les premières à être utilisées pour cette partie.

Après le pré-traitement, il faut donc découper l'image et pour cela, on appelle la fonction `image_splitting` du **get_letters.c** qui va appeler à son tour toutes les fonctions du fichier de sorte à faire un découpage idéal. Pour pouvoir récupérer correctement le résultat du découpage, nous avons ajouté deux nouvelles structures : une destinée à la grille de mots et une autre à la liste de mots. Les deux structures seront utilisées pour l'IA ainsi que le solver. La structure `grid` possède également une structure `letters` qui va associer à chaque lettre de la grille, le chemin pour la retrouver dans le dossier `grid` correspondant ainsi que ses coordonnées sur l'image.

Maintenant que l'on possède nos `grid` et `words`, nous pouvons appeler l'IA pour transformer chaque image en lettre. D'abord, on parcourt l'ensemble de la `grid` pour appeler l'IA dessus et créer un fichier texte semblable à notre grille. Si on fait cela dans un fichier texte, c'est pour pouvoir l'utiliser par la suite dans le solver qui lui lit la grille dans un fichier. On stocke aussi l'ensemble des mots dans une matrice.

La prochaine étape est d'appeler le solver. On va d'abord créer un array qui va nous permettre de stocker les coordonnées de chaque résultat. Ensuite, on parcourt notre matrice de mots et on appelle la solver en utilisant le fichier `grid` créé et les différents mots et ce tout en gardant le résultat dans l'array mentionné.

Une fois cela fait, il ne reste plus qu'à afficher le résultat. En utilisant les coordonnées stockées, on peut enfin tracer des lignes rouges sur notre image afin d'identifier chaque mot trouvé. Ce résultat est alors enregistré à la racine du projet sous le nom de "results.png". Un bouton back s'affiche également pour permettre de revenir au menu de démarrage et de refaire à nouveau la résolution ou l'entraînement.

Relier les codes entre eux n'a pas été une mince affaire. Il a fallu revenir sur chaque fichier, revoir le fonctionnement et même les modifier. Par exemple, il a fallu revenir sur toute la fin du `get_letters.c` pour initialiser correctement les structures nécessaires pour l'appel de l'IA. Aussi, puisque pour la première soutenance chaque fichier fonctionnait séparément, nous avons dû des fois réadapter les codes des anciennes main pour qu'elles s'accordent avec le `main.c`.

6 Bibliographie

1. WIKIPEDIA. *Connected-component labeling* [en ligne]. Wikipedia Foundation, 2024 [consulté le 19 oct. 2025]. Disponible sur : https://en.wikipedia.org/wiki/Connected-component_labeling
2. LA REVUE IA. *Clustering : les 3 méthodes à connaître* [en ligne]. 2023 [consulté le 11 oct. 2025]. Disponible sur : <https://larevueia.fr/clustering-les-3-metho>
3. GONZALEZ, Rafael C. et WOODS, Richard E. *Digital Image Processing*. Pearson.
4. GOODFELLOW, Ian, BENGIO, Yoshua et COURVILLE, Aaron. *Deep Learning (L'apprentissage profond)*. MIT Press, 2016.
5. GEEKSFORGEES. *Search a Word in a 2D Grid of characters* [en ligne]. [Consulté le 19 oct. 2025]. Disponible sur : <https://www.geeksforgeeks.org/search-a-word-in-a-2d-grid-of-characters/>
6. YOUTUBE. *[Titre de la vidéo]* [en ligne]. YouTube, [s.d.] [consulté le 20 oct. 2025]. Disponible sur : <https://www.youtube.com/watch?v=inF1x0-EYjU>
7. YOUTUBE. *[Titre de la vidéo]* [en ligne]. YouTube, [s.d.] [consulté le 20 oct. 2025]. Disponible sur : <https://www.youtube.com/watch?v=JX4Wvi6E8jQ>
8. STACK OVERFLOW. *How to render text in SDL2* [en ligne]. [s.d.] [consulté le 20 oct. 2025]. Disponible sur : <https://stackoverflow.com/questions/22886500/how-to-render-text-in-sdl2>
9. STACK OVERFLOW. *How to list files in a directory in a C program* [en ligne]. [s.d.] [consulté le 20 oct. 2025]. Disponible sur : <https://stackoverflow.com/questions/4204666/how-to-list-files-in-a-directory-in-a-c-program>

7 Conclusion

Pour cette deuxième et dernière soutenance, nous avons réussi à terminer entièrement le projet OCR dans les temps. Notre réseau de neurones est à la hauteur de nos attentes : un réseau stable, fiable et qui apprend efficacement. La finalisation du pré-traitement avec l'arrivée de la fonction d'auto-rotation permet de rendre totalement automatique le pré-traitement pour faciliter au mieux la détection et le découpage des lettres. L'implémentation de toutes ces fonctionnalités dans une interface graphique n'a pas été une tâche simple, mais le résultat obtenu permet notre projet OCR d'être plus intuitif, facile d'utilisation et fonctionnel. Malgré les difficultés rencontrées, principalement dans le développement du réseau de neurones ainsi que dans l'interface graphique, nous avons su évoluer dans le bon sens pour mener au mieux notre projet et pouvoir le finir dans les délais, avec la finition souhaitée. Nous sommes donc fiers de vous présenter ce projet terminé.